

DSA

1. Delete a Node in BST

The screenshot shows a LeetCode submission for the problem "Delete a Node in BST". The submission is accepted, with a runtime of 0 ms and memory usage of 47.62 MB. The code is written in Java and implements a recursive solution to delete a node from a binary search tree. The test result shows that the solution passed all three test cases.

Runtime: 0 ms, Beats 100.00%
Memory: 47.62 MB, Beats 18.32%

Code (Java):

```
1 /**
2  * Definition for a binary tree node.
3  * public class TreeNode {
4  *     int val;
5  *     TreeNode left;
6  *     TreeNode right;
7  *     TreeNode() {}
8  *     TreeNode(int val) { this.val = val; }
9  * }
10 */
11
12 class Solution {
13     public TreeNode deleteNode(TreeNode root, int key) {
14         if (root == null) {
15             return null;
16         }
17
18         if (key < root.val) {
19             root.left = deleteNode(root.left, key);
20         }
21         else if (key > root.val) {
22             root.right = deleteNode(root.right, key);
23         }
24         else {
25             if (root.left == null) {
26                 return root.right;
27             }
28             if (root.right == null) {
29                 return root.left;
30             }
31
32             TreeNode successor = root.right;
33             while (successor.left != null) {
34                 successor = successor.left;
35             }
36
37             root.val = successor.val;
38             root.right = deleteNode(root.right, key);
39         }
40         return root;
41     }
42 }
```

Testcase: Accepted, Runtime: 0 ms

Case 1: Input: root = [5,3,6,2,4,null,7], key = 3. Output: [5,4,6,2,null,null,7]

2. Check if a binary tree is a BST

The screenshot shows a LeetCode submission for the problem "Validate Binary Search Tree". The submission is accepted, with a runtime of 0 ms and memory usage of 45.08 MB. The code is written in Java and implements a recursive solution to validate a binary search tree. The test result shows that the solution passed both test cases.

Runtime: 0 ms, Beats 100.00%
Memory: 45.08 MB, Beats 65.65%

Code (Java):

```
1 /**
2  * Definition for a binary tree node.
3  * public class TreeNode {
4  *     int val;
5  *     TreeNode left;
6  *     TreeNode right;
7  *     TreeNode() {}
8  *     TreeNode(int val) { this.val = val; }
9  * }
10 */
11
12 class Solution {
13     public boolean isValidBST(TreeNode root) {
14         return validate(root, Long.MIN_VALUE, Long.MAX_VALUE);
15     }
16
17     public boolean validate(TreeNode root, long min, long max) {
18         if (root == null) {
19             return true;
20         }
21
22         if (root.val <= min || root.val >= max) {
23             return false;
24         }
25
26         return validate(root.left, min, root.val) &&
27             validate(root.right, root.val, max);
28     }
29 }
```

Testcase: Accepted, Runtime: 0 ms

Case 1: Input: root = [2,1,3]. Output: true

3. Find the kth smallest and kth largest elements in a BST.

Product Train

Open Repo

charitha08

Java Code

LeetCode

Search - Le

Kth Smallest

Validate Bin

Ask Gemini

leetcode.com/problems/kth-smallest-element-in-a-bst/submissions/2136848866/

Accepted

94 / 94 testcases passed

charitha_08 submitted at Sep 10, 2026 02:27

Analysis

Solution

Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime

0 ms | Beats 100.00%

Memory

46.75 MB | Beats 50.82%

Code

Java

```

1 /**
2  * Definition for a binary tree node.
3  * public class TreeNode {
4  *     int val;
5  *     TreeNode left;
6  *     TreeNode right;
7  *     TreeNode() {}
8  *     TreeNode(int val) { this.val = val; }
9  * }
10 */
11
12 class Solution {
13     int count = 0;
14     int ans = 0;
15     public int kthSmallest(TreeNode root, int k) {
16         inOrder(root,k);
17         return ans;
18     }
19     public void inOrder(TreeNode root, int k){
20         if(root == null) return;
21
22         inOrder(root.left,k);
23         count++;
24         if(count == k) {
25             ans = root.val;
26             return;
27         }
28         inOrder(root.right,k);
29     }
30 }

```

Testcase

Test Result

You must run your code first

Problem List

Description

Accepted

Editorial

Solutions

Submissions

All Submissions

Runtime

Memory

Code

Java

Testcase

Test Result

71. Second Minimum Node In a Binary Tree

4. Find whether the given BST is balanced or not.

leetcode.com/problems/balanced-binary-tree/submissions/213850171

Problem List

All Submissions

Accepted 278 / 278 testcases passed

charitha_08 Submitted at Sep 10, 2025 02:30

Unlock the Full LeetCode Experience

7583.25ms

0 ms Runtime Beats 100.00%

45.50 MB Memory Beats 59.75%

Runtime

Memory

0 ms 100.00%

45.50 MB 59.75%

75%

50%

25%

0%

1ms

2ms

3ms

4ms

100%

200%

300%

400%

Code

Java

```
1 //
2 // Definition for a binary tree node.
3 // public class TreeNode {
4 //     int val;
5 //     TreeNode left;
6 //     TreeNode right;
7 //     TreeNode() {}
8 //     TreeNode(int val) { this.val = val; }
9 // }
10
11 // View more
```

More challenges

104. Maximum Depth of Binary Tree

3319. Kth Largest Perfect Subtree Size in Binary Tree

3340. Check Balanced String

Write your notes here

Select related tags

0/5

Code

Java

```
15 //
16 // class Solution {
17 //     public boolean isBalanced(TreeNode root) {
18 //         return height(root) != -1;
19 //     }
20 //
21 //     public int height(TreeNode root) {
22 //         if (root == null) {
23 //             return 0;
24 //         }
25 //
26 //         int left = height(root.left);
27 //
28 //         if (left == -1) {
29 //             return -1;
30 //         }
31 //
32 //         int right = height(root.right);
33 //
34 //         if (right == -1) {
35 //             return -1;
36 //         }
37 //
38 //         if (Math.abs(left - right) > 1) {
39 //             return -1;
40 //         }
41 //
42 //         return Math.max(left, right) + 1;
43 //     }
44 // }
```

Saved

Ln 42, Col 2

Testcase

Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

root = [3,9,20,null,null,15,7]

Output

true

Expected

true

